CUDA Ghost in the Shell (CGinS): Automated LLM-Driven CUDA Kernel Generation for PyTorch Models

Joshua Brodsky
University of Michigan
joshbrod@umich.edu

Arnav Shah
University of Michigan
sarnav@umich.edu

Abstract

The popularity of PyTorch's eager mode execution facilitates flexibility but often precludes the global graph-level optimizations available to compiled backends, making performance dependent on individual kernel efficiency. To address this bottleneck, we introduce CUDA Ghost in the Shell (CGinS), a framework that leverages Large Language Models (LLMs) to automate the generation and optimization of CUDA kernels. Unlike traditional static compilers, CGinS utilizes runtime profiling to capture real-world execution contexts (e.g. input tensor shapes and types) to prompt LLMs for hardware-aware kernel creation.

The system employs a validation-guided feedback loop where compilation and runtime errors are fed back to the model, allowing for iterative self-correction until convergence. Following correctness validation, a single-shot hardware optimization pass targets specific GPU architectural constraints (e.g., NVIDIA RTX 3090) to maximize throughput and memory efficiency. Evaluations across standard deep learning workloads, including ResNet-50 and DistilBERT, demonstrate a 1.69x speedup (40.6% reduction) in total GPU time compared to baseline PyTorch operators, with significant latency reductions in dominant operations like linear and dropout. These results validate the feasibility of using LLMs for low-level systems programming when supported by rigorous introspection and validation mechanisms.

1 Introduction

The rapid expansion of the Artificial Intelligence (AI) and Large Language Model (LLM) landscape has made computational efficiency a critical bottleneck for both training and inference workloads. A popular framework for architecting, training, and running Machine Learning (ML) models is PyTorch. A key driver of its popularity stems from PyTorch's highly Pythonic imperative execution through eager mode. Since eager mode executes operators one at a time through Python, the runtime cannot perform many of the global graph-level optimizations that compiled PyTorch backends (e.g., torch.compile with Inductor or XLA) can apply. As a result, performance in eager mode depends primarily on the efficiency of individual kernel implementations. The system cannot apply global optimizations such as operator fusion, kernel tiling, memory planning, or graph-level scheduling that compiled backends can introduce.

Due to each op being run one at a time, a great deal of effort has been put into extracting a higher performance of each individual op. This historically has stemmed from manual kernel creation or compiler-based optimizations, however, each method has a significant issue. Compiler-based optimizations suffer from a massive search space that continues to grow and change as new model architectures arise. Manual kernel creation is tedious and requires industry experts with hardware and architecture specific knowledge to produce performant kernels. Additionally, many of PyTorch eager mode's ops utilize closed-source libraries such as cuDNN/cuBLAS so kernel developers do not have a baseline to reference when writing their own kernels.

Within the last few years, there has been a rise in use of LLMs for programming through both usage of base-models and Agentic IDEs. Models such as Anthropic's Claude, Google's Gemini, and OpenAI's ChatGPT have been competitive with professional programmers, achieving high performance on benchmarks such as SWE-Bench. Additionally Agentic IDEs/Terminal apps such as Anthropic's Claude Code, Google's Antigravity, and Codeium's Cursor have implemented tools to both generate and validate code, streamlining the process of code iteration.

Due to both the success that these LLMs and Agentic tools have in tackling coding problems and necessity to optimize individual ML ops, we evaluate the efficacy of using LLMs as a compiler to generate and optimize kernels matching

behavior of PyTorch functions. Unlike traditional compilers that rely on static analysis, our approach utilizes runtime profiling to capture PyTorch API calls (ATen, cuBLAS/cuDNN) and contextual data (e.g., input tensor shapes and types) from real-world models to prompt LLMs to generate hardware-aware CUDA kernels directly. However, due to the frequent initial failures that result from zero-shot code generation, we introduce a pipeline that incorporates a validation-guided feedback loop where compilation, runtime, and correctness errors are fed back to the model for iterative development. We observe that while initial attempts rarely pass, this iterative refinement allows the model to self-correct, typically converging on correct and performant kernels within 2-4 iterations.

In this paper, we present CUDA Ghost in the Shell (CGinS) to evaluate the feasibility of an LLM-guided optimization pipeline. CGinS is evaluated through generating kernels for the PyTorch ops of standard deep learning workloads, across four models (ResNet-50, DistiliBERT, Swin-Base, ConvNeXt-Base) and model types (image classification and text classification). We specifically address the performance of the pipeline by grading the pipeline on its ability to generate ops that are both correct and perform competitively with PyTorch. Initial findings support our proposed pipeline's efficacy with all 8 PyTorch ops successfully having an associated LLM-generated CUDA kernel whose correctness is validated and execution time (`torch.cuda.Event`) is achieving within 20% on average of PyTorch's latency of highly-optimized vendor kernels.

2 Methodology

In this section, we will describe the pipeline for generating and optimizing CUDA kernels that implement the behavior of high-level PyTorch operators. The three main stages of the pipeline can be broken down into collecting runtime data from real-world models, generating kernels with a validation loop, and generated kernels for a specific hardware architecture.

2.1 Runtime Data Collection

To determine which PyTorch ops to target in our initial generation batch and to enable a thorough validation stage, we profiled PyTorch op calls from real-world models (Figure 1). For our data collection section of the CGinS pipeline, we run four models spanning classification of text and images with samples of training data to simulate real workloads (ResNet-50, DistilBERT, Swin-Base, and ConvNeXt-Base).

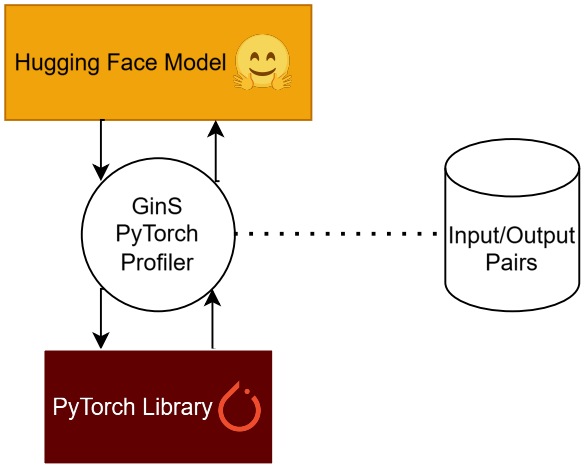

Figure 1: Data Collector

Before running these models, we register instrumentation hooks to each high-level PyTorch op in the `torch.nn.functional` function list (i.e. linear, relu, conv2d, etc.). Each hook is an intercept call that records the positional arguments and keyword arguments of the function call, runs the function with the input, records the output produced, and then returns the output to function caller.

Once we add this hook, we can run the model and store the input-output pairs associated with each function to a PT file for later use. This allows us to retain values of non-primitive variable types (e.g. tensor) of real world models.

To ensure deterministic ordering of parameters, we canonicalize all possible positional arguments as keyword arguments, storing the PyTorch "default" value if a given parameter is not provided.

We later use the profiled input/output pairs for correctness validation post-kernel-generation.

2.2 Kernel Generation

Once the initial dataset of PyTorch functions is collected, we begin our generation and validation loop (Figure 2). This loop is applied to each high-level PyTorch op independently. The loop can be divided into three main sections: Monitor, Generator, and Verifier.

2.2.1 Monitor

The pipeline begins with the Monitor, a dynamic introspection module designed to profile individual PyTorch operations and extract their underlying execution behavior. By leveraging the PyTorch profiler within a controlled warmup-and-

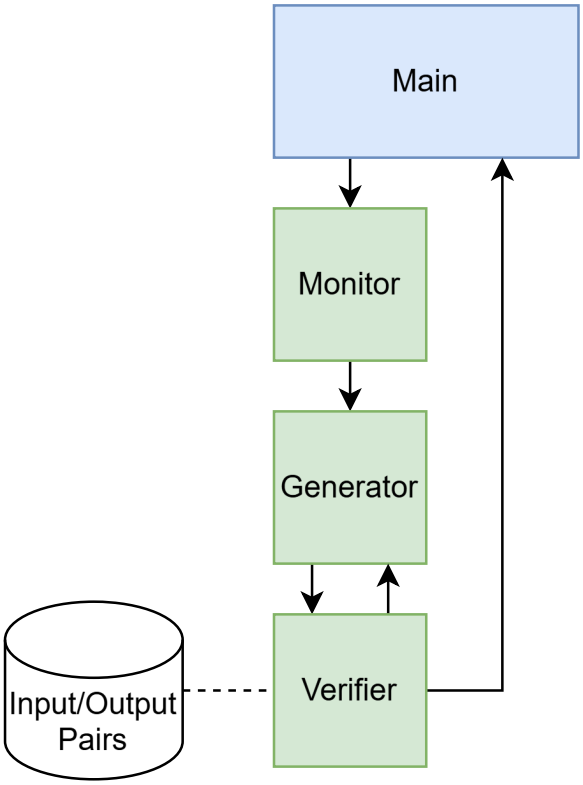

Figure 2: Generation and Validation Pipeline

execute schedule, the Monitor captures runtime events, specifically isolating high-level `ATen` operators (e.g., `aten::mm`) and correlating them with their corresponding low-level CUDA kernel launches.

This execution data is synthesized into a structured context string to later provide the LLM with ground-truth insights into PyTorch's implementation of handling GPU execution strategies.

2.2.2 Generator

The Generator synthesizes the collected execution context into a comprehensive prompting strategy. It dynamically constructs the user prompt by analyzing the runtime trace to deduce argument data types and tensor shapes, while simultaneously embedding PyTorch profiler data to elucidate the underlying `ATen` operations and closed-source CUDA kernels.

To enforce implementation constraints, the Generator appends a strict system prompt dictating the output format:

1. **Single File:** The entire kernel, including overhead, must be contained within a single `kernel.cu`.

2. **Delimiters:** Each section of the generated `kernel.cu` must contain custom delimiters (e.g. `[START kernel.cu]`) to separate each section (e.g. include statements, literal kernel functions, overhead) for easier parsing later.

3. **Consistent Overhead:** Each `kernel.cu` file requires the overhead to be executed by a Python `module.launch` call. This means that each file must have a `launch()` overhead function to act as a standardized `C++` host wrapper to act as a foreign function interface for maintaining precise type-mapping between Python/PyTorch types and C++ equivalents.

It then merges the System Prompt with the Task Prompt into a model-provider-agnostic conversation history data structure, dispatches the request, and employs utility functions to extract and sanitize the resulting CUDA kernel code.

2.2.3 Verifier

The Verifier orchestrates the validation phase of the generation pipeline, enforcing strict standards for compilation and numerical precision. It must pass several tests before it has been validated.

1. **Compilation:** The process begins with an on-the-fly compilation attempt using PyTorch's `load_inline`.

2. **Runtime:** Next, the kernel that was compiled was run on a ground-truth known input from the data collection phase.

3. **Correctness:** Once the kernel is successfully, compiled and runtime provided and output, we compare the runtime output to our ground-truth output via `torch.allclose`.

In the event of any of the above stages failing, depending on error type, exception traces or metadata is analyzed by a secondary helper LLM to provide actionable debugging feedback. The secondary helper LLM is incorporated to prevent context window flooding.

This feedback is then added to the conversation history as a Refinement Prompt for subsequent iterations. This self-correcting cycle is capped at five attempts to prevent excessive token consumption on kernels that fail to converge.

If a kernel satisfies all validation criteria, it is persisted for deployment; otherwise, the attempt is logged as a failure, and the pipeline advances to the next operator.

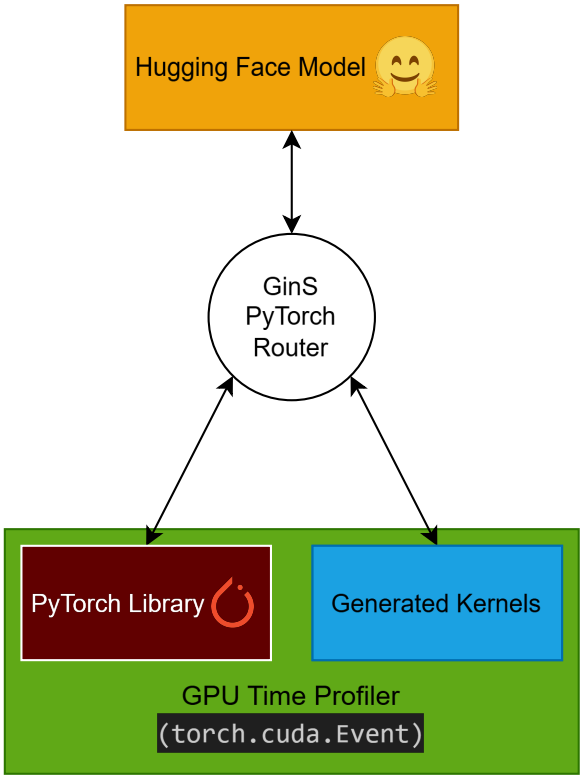

Figure 3: Measure CUDA event

2.3 Hardware-Aware Optimization

After establishing correctness through validation, we perform a hardware-aware optimization pass aimed solely at improving runtime performance on a target GPU. Unlike the correctness stage, which may involve multiple attempts to obtain a valid kernel, the hardware optimization stage is intentionally single-shot. Given a validated CUDA kernel, the language model is prompted exactly once to produce an optimized variant under explicit hardware constraints corresponding to the NVIDIA RTX 3090.

The prompt encodes architectural details such as SM count, shared memory limits, register file size, warp width, and occupancy-related constraints. The model is instructed to preserve the kernel's interface and semantics while optimizing for throughput, memory efficiency, and occupancy. Importantly, this step does not attempt to correct errors or search across multiple variants. The goal is to isolate the effect of hardware-aware prompting itself, rather than conflating it with iterative search, tuning heuristics, or reinforcement learning.

Although the optimization is single-shot, the resulting kernel is still subjected to the same validation test suite used earlier to ensure functional equivalence. Kernels that fail validation are discarded, while those that pass are forwarded to compilation and profiling. This preserves correctness guarantees while allowing performance gains to be attributed specifically to hardware-aware generation.

3 Results

Performance is evaluated by running full end-to-end inference on three image classification models (ResNet-50, Swin-Base, and ConvNeXt-Base) and one text classification model (DistilBERT). These models collectively exercise a wide range of tensor shapes and operator invocation patterns.

Kernel execution time is measured using CUDA events to isolate GPU-side latency. Each experiment is repeated 11 times; the first iteration is discarded as a GPU warm-up, and the remaining 10 iterations are used for performance measurement.

3.1 Kernel-Level Performance

Table 1 reports average latency for the baseline (non-hardware-aware) kernels and the hardware-aware optimized kernels. Across all workloads, five operators dominate measured time: `linear`, `dropout`, `embedding`, `GELU`, and `ReLU`.

Hardware-aware optimization improves the primary bottlenecks, but not uniformly across every operator. In particular, `linear` and `dropout` see large latency reductions, while `embedding` and `ReLU` become slightly slower in this workload.

Overall, the total measured GPU time across these kernels drops from 1981.8275 ms to 1176.2889 ms, a 1.69× speedup (40.6% reduction). Most of this improvement is driven by `linear`, which is invoked 8,360 times and dominates total time.

As shown in Table 1, `linear` average latency decreases from 0.2109 ms to 0.1208 ms per call, corresponding to a 1.75× speedup. In total time, `linear` drops from 1763.4109 ms to 1009.7806 ms, accounting for 753.6303 ms of the overall 805.5386 ms GPU-time reduction. `dropout` also improves substantially, with average latency halving from 0.0601 ms to 0.0299 ms (2.01×), reducing total time by 53.0318 ms.

Other operators show mixed behavior. `GELU` improves modestly (0.0502 ms → 0.0432 ms, 1.16×), while `embedding` regresses (0.0852 ms → 0.1060 ms, 0.80×) and `ReLU` regresses slightly (0.0410 ms → 0.0465 ms, 0.88×). These regressions suggest that single-shot hardware prompting does not reliably improve already bandwidth-

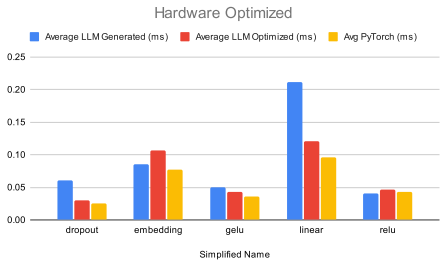

Figure 4: Average per-call latency before and after hardware-aware optimization.

efficient or library-tuned primitives, and that additional tuning/selection would be needed for consistently best-per-op results.

Figure 4 visualizes these effects by comparing average per-call latency for each kernel before and after hardware-aware optimization.

4 Related Works

While developing CGinS, we identified several closely related projects that informed our design choices, most notably Scaling Intelligence's KernelBench and AMD's GEAK. Both systems leverage large language models to generate optimized GPU kernels for machine learning workloads; however, their goals and system designs differ substantially from CGinS.

4.1 KernelBench

KernelBench is a benchmark developed by Scaling Intelligence at Stanford University for evaluating the ability of large language models (LLMs) to generate efficient CUDA kernels [2]. It provides a standardized evaluation framework with fixed problem definitions, input/output specifications, and a performance measurement harness, enabling controlled comparison of LLMs on GPU kernel generation tasks.

CGinS draws inspiration from two primary aspects of KernelBench. First, CGinS adopts the exposure of hardware-specific information to the LLM to encourage performance-aware kernel generation. Secondly, CGinS adopts KernelBench's approach to Python-CUDA interoperability, allowing for the direct invocation of generated kernels on PyTorch tensors without data copying or serialization overhead.

On the other hand CGinS implements several features that KernelBench does not offer. These include monitoring real PyTorch executions, capturing concrete runtime properties such as input shapes and strides, and iteratively generating and validating custom CUDA kernels to replace existing operators. Unlike KernelBench's single-pass or best-of-N evaluation on fixed synthetic inputs, CGinS employs an iterative feedback loop that incorporates compilation and correctness errors, and it validates both correctness and performance using the actual inputs observed during execution.

Despite these similarities, the two systems differ fundamentally in purpose. KernelBench is an evaluation benchmark, whereas CGinS is an optimization pipeline designed for deployment. Thus, while both systems involve LLM-generated CUDA kernels, KernelBench does not address the direct problem targeted by CGinS.

4.2 GEAK

GEAK explores LLM-generated GPU kernels targeting AMD's ROCm platform and serves as a natural comparison point [3]. However, there are key methodological differences that affect both correctness and evaluation.

First, GEAK evaluates kernels using synthetic inputs and TritonBench-derived[1] micro-benchmarks rather than real model executions. Many kernels that may pass synthetic tests fail under real workloads due to mismatched tensor layouts, boundary conditions, or semantic assumptions.

Additionally, GEAK reports performance using isolated operator benchmarks rather than end-to-end inference. Our results demonstrate that kernel-level improvements, particularly for dominant operators such as `linear`, translate directly into reduced GPU time during real model execution.

Overall, while GEAK demonstrates the feasibility of LLM-based kernel generation, our results indicate that hardware-aware prompting, real-workload validation, and feedback-driven iteration are essential for reliable and practically meaningful performance gains.

5 Limitations

While CGinS demonstrates promising performance improvements, several limitations remain.

First, kernel correctness and convergence were highly sensitive to the choice of language model. Early experiments with smaller local models (e.g., LLaMA-3.2-8B) failed to produce correct CUDA kernels within the validation loop. Mid-sized models (e.g. Gemini 2.5 Flash) occasionally generated correct implementations, but results were inconsistent across runs. In contrast, larger foun-

Table 1: Average per-call latency (ms) for LLM-generated, optimized, and PyTorch baseline kernels on an RTX 3090.

Kernel Name	Avg LLM Generated (ms)	Avg LLM Optimized (ms)	Avg PyTorch (ms)
dropout	0.0601	0.0299	0.0254
embedding	0.0852	0.1060	0.0766
gelu	0.0502	0.0432	0.0355
linear	0.2109	0.1208	0.0957
relu	0.0410	0.0465	0.0430

dation models (e.g., Claude Sonnet 4.5) consistently met correctness requirements, typically producing valid kernels within the five iterative refinement attempt limit. This reliance on larger models limits the accessibility and deployability of CGinS and suggests that improvements in smaller model reasoning or distillation may be necessary for broader adoption.

Second, CGinS currently applies a limited form of hardware-aware optimization. After generating a zero-shot, correctness-validated kernel, the system performs a single optimization pass by augmenting the prompt with static GPU metadata (e.g., device name, architecture, warp size). This optimization step is non-iterative and does not incorporate performance feedback beyond the initial measurement. Future work could extend this approach by introducing an iterative performance optimization loop in which the model repeatedly analyzes profiling results and proposes targeted kernel-level optimizations.

Finally, the current scope of CGinS is restricted to individual operators with general-purpose usage assumptions. In practice, real-world models exhibit structured and predictable characteristics, such as fixed input shapes, sparsity patterns, and stable hyperparameters, which could inform more aggressive specialization. Extending CGinS to exploit these properties-through operator fusion, kernel consolidation, or end-to-end forward-pass generation-represents an important direction for future research. Additionally, integrating CGinS into an real-time inference setting, where kernels are continuously re-optimized based on observed workloads, may further improve performance over time.

6 Conclusion

We presented CUDA Ghost in the Shell (CGinS), a framework that automates the engineering of high-performance CUDA kernels through an iterative, LLM-driven validation pipeline. Unlike static compilation methods, CGinS leverages runtime profiling and feedback loops to generate kernels that adapt to specific model architectures and data types. Our results indicate that this approach effectively bridges the gap between Pythonic eager execution and hardware-level optimization, producing kernels that compete closely with established vendor libraries.

These findings suggest that LLMs are viable candidates for handling low-level systems programming tasks when constrained by rigorous validation mechanisms. While currently limited to individual operators, this methodology lays the groundwork for future systems capable of end-to-end model optimization. By reducing the barrier to entry for custom kernel development, CGinS offers a promising path toward more accessible and efficient AI acceleration.

References

[1] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, WangHaojie Wang, Jianrong Wang, Xu Han, et al. Tritonbench: Benchmarking large language model capabilities for generating triton operators. *arXiv preprint arXiv:2502.14752*, 2025.

[2] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2501.01318*, 2025.

[3] Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. Geak: Introducing triton kernel ai agent & evaluation benchmarks. *arXiv preprint arXiv:2507.23194*, 2025.